



Assignment IV

1. Find Eigenvalues using Colab

Find the eigenvectors and eigenvalues of σ_x using colab. You may use this code. But please ask yourself, which basis are we using when we express σ_x in the following form?

```
import numpy as np
sigmax=np.array([[0,1],[1,0]])
print(sigmax)
eigenvalues, eigenvectors = np.linalg.eig(sigmax)
print(eigenvalues)
print(eigenvectors)
```

2. Construct Projection Operator

Now, use hand calculation, derive the projection operators to project to the eigenstates of σ_z .

3. Use Projection Operator to find Measurement probability

Use the projector equation to find the probability of measuring spin up and down when it is applied to the eigenvectors of σ_x . Hints: you should get 50% for all of them.

4. Vector normalization by hand and using python 2

Check Problem 3 using co-lab. Hints:

- Use np.transpose for transpose
- Use np.dot for multiplication, including outer product
- Use np.linalg.eig to find eigenvalues

5. Orthonormality of Unitary Matrix Row Vectors

Prove that the row vectors of a unitary matrix are orthonormal.

6. Finding the Determinants of Pauli Matrices by Hand

Find the determinants of σ_x , σ_y , and σ_z . What do you notice?

7. Finding the Determinants of Pauli Matrices using Colab

Confirm your answers in Problem 6 using Colab. This is a sample code.

```
import numpy as np
M=np.array([[1,0],[0,-1]])
print(np.linalg.det(M))
```

1. Find Eigenvalues using Colab

Find the eigenvectors and eigenvalues of σ_x using colab. You may use this code. But please ask yourself, which basis are we using when we express σ_x in the following form?

```
import numpy as np
sigmax=np.array([[0,1],[1,0]])
print(sigmax)
eigenvalues, eigenvectors = np.linalg.eig(sigmax)
print(eigenvalues)
print(eigenvectors)
```

```
import numpy as np

sigmax = np.array([[0, 1], [1, 0]])
print(sigmax)
eigenvalues, eigenvectors = np.linalg.eig(sigmax)
print(eigenvalues)
print(eigenvectors)
```

```
● (quantum-computing) (base) pupip
antum-computing/.venv/bin/python
[[0 1]
 [1 0]]
[ 1. -1.]
[[ 0.70710678 -0.70710678]
 [ 0.70710678  0.70710678]]
○ (quantum-computing) (base) pupip
```

2. Construct Projection Operator

Now, use hand calculation, derive the projection operators to project to the eigenstates of σ_x .

$$\sigma_z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad |0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad (\lambda=1), \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad (\lambda=-1)$$

The projector onto eigenstate $|\phi\rangle$: $P_\phi = |\phi\rangle\langle\phi|$

$$P_0 = |0\rangle\langle 0| = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \quad (\lambda=1)$$

$$P_1 = |1\rangle\langle 1| = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \quad (\lambda=-1)$$

Spectral decomposition form

$$\text{form } \sigma_z = (+1)P_0 + (-1)P_1 \quad \text{and} \quad P_0 + P_1 = I$$

$$\therefore \text{ Soln: } P_0 = \frac{I + \sigma_z}{2}, \quad P_1 = \frac{I - \sigma_z}{2} \quad \text{✓}$$

3. Use Projection Operator to find Measurement probability

Use the projector equation to find the probability of measuring spin up and down when it is applied to the eigenvectors of σ_x . Hints: you should get 50% for all of them.

We'll measure in the z -basis (Spin up $|0\rangle$, Spin down $|1\rangle$)

Projectors onto $|0\rangle, |1\rangle$: $P_0 = |0\rangle\langle 0| = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$, $P_1 = |1\rangle\langle 1| = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$

Eigenvectors of σ_z : $|+\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $|-\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$

Born rule with projectors for state $|\psi\rangle$: $P_{\text{up}} = \langle\psi|P_0|\psi\rangle$, $P_{\text{down}} = \langle\psi|P_1|\psi\rangle$

For $|+\rangle$:

$$\therefore P_{\text{up}} = \langle + | P_0 | + \rangle = \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \end{bmatrix} \right) \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) = \frac{1}{2} \quad \#$$

$$\therefore P_{\text{down}} = \langle + | P_1 | + \rangle = \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \end{bmatrix} \right) \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) = \frac{1}{2} \quad \#$$

For $|-\rangle$:

$$\therefore P_{\text{up}} = \langle - | P_0 | - \rangle = \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \end{bmatrix} \right) \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} \right) = \frac{1}{2} \quad \#$$

$$\therefore P_{\text{down}} = \langle - | P_1 | - \rangle = \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \end{bmatrix} \right) \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} \right) = \frac{1}{2} \quad \#$$

4. Vector normalization by hand and using python 2

Check Problem 3 using co-lab. Hints:

Use `np.transpose` for transpose

Use `np.dot` for multiplication, including outer product

Use `np.linalg.eig` to find eigenvalues

```
import numpy as np

# Basis projectors for  $\sigma_z$ 
P0 = np.array([[1, 0], [0, 0]], dtype=complex) #  $|0\rangle\langle 0|$ 
P1 = np.array([[0, 0], [0, 1]], dtype=complex) #  $|1\rangle\langle 1|$ 

#  $\sigma_x$  and its eigenvectors (columns)
sigmax = np.array([[0, 1], [1, 0]], dtype=complex)
evals, evects = np.linalg.eig(sigmax)
# evects[:,0] ~  $|+\rangle$ , evects[:,1] ~  $|-\rangle$  (up to a global phase)

# Ensure column vectors (kets)
ket_plus = evects[:, [0]] # shape (2,1)
ket_minus = evects[:, [1]] # shape (2,1)

# Bras are conjugate-transposes
bra_plus = np.transpose(ket_plus.conj()) #  $\langle +|$ 
bra_minus = np.transpose(ket_minus.conj()) #  $\langle -|$ 

# Probabilities via projectors:  $\langle \psi | P | \psi \rangle$ 
p_up_plus = np.dot(bra_plus, np.dot(P0, ket_plus)) # scalar 1x1
p_down_plus = np.dot(bra_plus, np.dot(P1, ket_plus))
p_up_minus = np.dot(bra_minus, np.dot(P0, ket_minus))
p_down_minus = np.dot(bra_minus, np.dot(P1, ket_minus))

print("Eigenvalues of  $\sigma_x$ :", evals)
print("P(up) for  $|+\rangle$ :", np.real_if_close(p_up_plus[0, 0]))
print("P(down) for  $|+\rangle$ :", np.real_if_close(p_down_plus[0, 0]))
print("P(up) for  $|-\rangle$ :", np.real_if_close(p_up_minus[0, 0]))
print("P(down) for  $|-\rangle$ :", np.real_if_close(p_down_minus[0, 0]))
```

```
• (quantum-computing) (base) pupipatsing
  antum-computing/.venv/bin/python /User
  Eigenvalues of  $\sigma_x$ : [ 1.+0.j -1.+0.j]
  P(up) for  $|+\rangle$ : 0.4999999999999998
  P(down) for  $|+\rangle$ : 0.5000000000000001
  P(up) for  $|-\rangle$ : 0.4999999999999999
  P(down) for  $|-\rangle$ : 0.4999999999999999
○ (quantum-computing) (base) pupipatsing
```

5. Orthonormality of Unitary Matrix Row Vectors

Prove that the (row vectors of a unitary matrix) are (orthonormal.)

Orthonormal:

1. Orthogonality
2. Normalization

$$\text{Let } U = \begin{bmatrix} r_1^T & r_2^T & \dots & r_n^T \end{bmatrix}$$

r_i is a row vector

The condition $U^\dagger U = I$ means:

$$U^\dagger U = \begin{bmatrix} r_1 \\ r_2 \\ \vdots \\ r_n \end{bmatrix} \begin{bmatrix} r_1^* & r_2^* & r_3^* & \dots & r_n^* \end{bmatrix} = I$$

$$\langle r_i | r_i \rangle = 1 : \text{Normalized (length 1)}$$

Off-diagonal: $\langle r_1 | r_2 \rangle = 0$, $\langle r_1 | r_3 \rangle = 0$, ... : All row vec are Orthogonal to each other.

∴ Each row vec is Normalized and Orthogonal to each other. ✖

Unitary matrix

33 languages

Article Talk

Read Edit View history Tools

From Wikipedia, the free encyclopedia

For matrices with orthogonality over the real number field, see [orthogonal matrix](#). For the restriction on the allowed evolution of quantum systems that ensures the sum of probabilities of all possible outcomes of any event always equals 1, see [unitarity](#).

In [linear algebra](#), an [invertible complex square matrix](#) *U* is **unitary** if its [matrix inverse](#) U^{-1} equals its [conjugate transpose](#) U^* , that is, if

$$U^* U = U U^* = I,$$

where *I* is the [identity matrix](#).

In [physics](#), especially in [quantum mechanics](#), the conjugate transpose is referred to as the [Hermitian adjoint](#) of a matrix and is denoted by a [dagger](#) (†), so the equation above is written

$$U^\dagger U = U U^\dagger = I.$$

A complex matrix *U* is **special unitary** if it is unitary and its [matrix determinant](#) equals 1.

For [real numbers](#), the analogue of a unitary matrix is an [orthogonal matrix](#). Unitary matrices have significant importance in quantum mechanics because they preserve [norms](#), and thus, [probability amplitudes](#).

6. Finding the Determinants of Pauli Matrices by Hand

Find the determinants of σ_x , σ_y , and σ_z . What do you notice?

$$\left. \begin{aligned} \det(\sigma_x) &= \begin{vmatrix} 0 & 1 \\ 1 & 0 \end{vmatrix} = -1 \\ \det(\sigma_y) &= \begin{vmatrix} 0 & -i \\ i & 0 \end{vmatrix} = 1 \\ \det(\sigma_z) &= \begin{vmatrix} 1 & 0 \\ 0 & -1 \end{vmatrix} = -1 \end{aligned} \right\} \begin{aligned} \det(\sigma_x) &= \det(\sigma_z) = -1 \\ \det(\sigma_y) &= 1 \end{aligned} \left\{ \begin{aligned} \pm 1 \\ \text{Same as} \\ \text{Eigenvalue} \end{aligned} \right. \quad \#$$

7. Finding the Determinants of Pauli Matrices using Colab

Confirm your answers in Problem 6 using Colab. This is a sample code.

```
import numpy as np
M=np.array([[1,0],[0,-1]])
print(np.linalg.det(M))
```

```
import numpy as np

sigmax = np.array([[0, 1], [1, 0]])
sigmay = np.array([[0, -1j], [1j, 0]])
sigmaz = np.array([[1, 0], [0, -1]])

print("det(σ_x) =", np.linalg.det(sigmax))
print("det(σ_y) =", np.linalg.det(sigmay))
print("det(σ_z) =", np.linalg.det(sigmaz))
```

```
● (quantum-computing) (base) pupip
antum-computing/.venv/bin/python
det(σ_x) = -1.0
det(σ_y) = (-1+0j)
det(σ_z) = -1.0
○ (quantum-computing) (base) pupip
```

8. Proof of Unitarity

For an orthonormal set of basis, $|v_i\rangle$, we have $\langle v_i | v_j \rangle = \delta_{i,j}$. We transform it using a matrix A , i.e. they become $A|v_i\rangle$. Prove that if after the transformation, they are still orthonormal, then A is unitary. Hints: Express the basis vectors in column form and each of them has only 1 entry. Express A in its matrix form. The fact that after the transformation, they are still orthonormal implies that the columns of A are orthonormal and thus A is unitary.

Soln :

$$\begin{aligned} \bullet \langle v_i | v_j \rangle &= \delta_{ij} \\ \bullet \langle v'_i | v'_j \rangle &= \langle A v_i | A v_j \rangle \\ &= v_i^\dagger A^\dagger A v_j = \delta_{ij} \end{aligned}$$


$$\downarrow$$
$$\therefore A^\dagger A = I$$

$\therefore A$ is Unitary

$\therefore$ After transformation, preserve Orthonormality, A is unitary. ~~*~~